LAPORAN PRAKTIKUM
Implementasi Function dan Trigger

Tugas Mandiri
Rekaya Arsitektur dan Basis Data

Dosen Pengampu
Cecep Kurnia Sastradipraja, S.Kom., M.Kom.

Disusun Oleh:
Nurul Karimah 2507065

Sistem Informasi B

INSTITUT TEKNOLOGI GARUT
PROGRAM STUDI SISTEM INFROMASI
2026

1. Deskripsi Kasus dan Rancangan Solusi

Studi kasus yang digunakan adalah sistem penjualan toko elektronik dengan basis data praktikum_function_trigger yang terdiri atas tabel produk, penjualan, dan log_stok. Pada tugas mandiri ini, solusi dikembangkan dengan skenario baru, yaitu perhitungan pajak, perhitungan total akhir setelah diskon, pencatatan setiap perubahan stok produk, dan validasi perubahan harga produk.

Rancangan solusi yang dikembangkan ditunjukkan pada tabel berikut.

Objek	Jenis	Rancangan
hitung_pajak	Function	Menerima total transaksi dan mengembalikan pajak sebesar 11% dari total.
hitung_total_akhir	Function	Menerima total transaksi dan persentase diskon, lalu mengembalikan Total - (Total x Diskon / 100).
catat_perubahan_stok	Trigger AFTER UPDATE pada produk	Jika stok berubah, mencatat ID produk, stok awal (OLD.stok), stok akhir (NEW.stok), keterangan, dan waktu ke tabel log_stok.
validasi_harga_produk	Trigger BEFORE UPDATE pada produk	Menolak perubahan harga menjadi nol atau negatif dengan SIGNAL SQLSTATE '45000'.

Pola yang digunakan mengikuti materi praktikum, yaitu function dengan parameter dan RETURN, percabangan IF, penggunaan OLD dan NEW pada trigger, SIGNAL SQLSTATE untuk validasi, serta pencatatan log pada tabel log_stok.

Lingkungan praktikum: layanan MySQL dijalankan melalui DBngin (MySQL 9.7.1, port 33011) dan perintah SQL dijalankan melalui DBeaver.



Gambar 1. DBngin: instance MySQL aktif (port 33011)

2. Kode SQL Lengkap

Kode SQL berikut dijalankan melalui DBeaver. Bagian A membuat function dan trigger, sedangkan bagian B berisi pengujian.

```
-- ===== BAGIAN A: PEMBUATAN FUNCTION DAN TRIGGER =====
USE praktikum_function_trigger;

-- Reset stok Mouse ke 25 SEBELUM trigger dibuat (supaya tidak ikut tercatat di log)
UPDATE produk SET stok = 25 WHERE id_produk = 2;

DELIMITER //

-- Tugas 1: function pajak 11%
CREATE FUNCTION hitung_pajak(
    p_total DECIMAL(14,2)
)
```

```
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_total * 0.11;
END //

-- Tugas 2: function total akhir
CREATE FUNCTION hitung_total_akhir(
   p_total DECIMAL(14,2),
   p_persen_diskon DECIMAL(5,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_total - (p_total * p_persen_diskon / 100);
END //

-- Tugas 3: trigger AFTER UPDATE (log perubahan stok)
CREATE TRIGGER catat_perubahan_stok
AFTER UPDATE ON produk
FOR EACH ROW
BEGIN
   IF OLD.stok <> NEW.stok THEN
      INSERT INTO log_stok (id_produk, stok_awal, stok_akhir, keterangan)
      VALUES (NEW.id_produk, OLD.stok, NEW.stok, 'Perubahan stok produk');
   END IF;
END //

-- Tugas 4: trigger BEFORE UPDATE (validasi harga)
CREATE TRIGGER validasi_harga_produk
BEFORE UPDATE ON produk
FOR EACH ROW
BEGIN
   IF NEW.harga <= 0 THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Harga produk harus lebih dari nol';
   END IF;
END //

DELIMITER ;

-- ===== BAGIAN B: PENGUJIAN =====
-- Tugas 1
SELECT 2000000 AS total_transaksi,
      hitung_pajak(2000000) AS pajak;

SELECT 5000000 AS total_transaksi,
      hitung_pajak(5000000) AS pajak;

-- Tugas 2
SELECT 3000000 AS total_transaksi,
      15 AS persen_diskon,
      hitung_total_akhir(3000000, 15) AS total_akhir;

-- Tugas 3
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 2;
```

```
UPDATE produk SET stok = 30 WHERE id_produk = 2;

SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 2;

SELECT * FROM log_stok WHERE id_produk = 2 ORDER BY id_log;

-- Tugas 4: skenario A (diterima)
UPDATE produk SET harga = 200000 WHERE id_produk = 2;

SELECT id_produk, nama_produk, harga FROM produk WHERE id_produk = 2;

-- Tugas 4: skenario B (ditolak)
UPDATE produk SET harga = 0 WHERE id_produk = 2;

SELECT id_produk, nama_produk, harga FROM produk WHERE id_produk = 2;

-- Pemeriksaan objek
SHOW FUNCTION STATUS WHERE Db = 'praktikum_function_trigger';

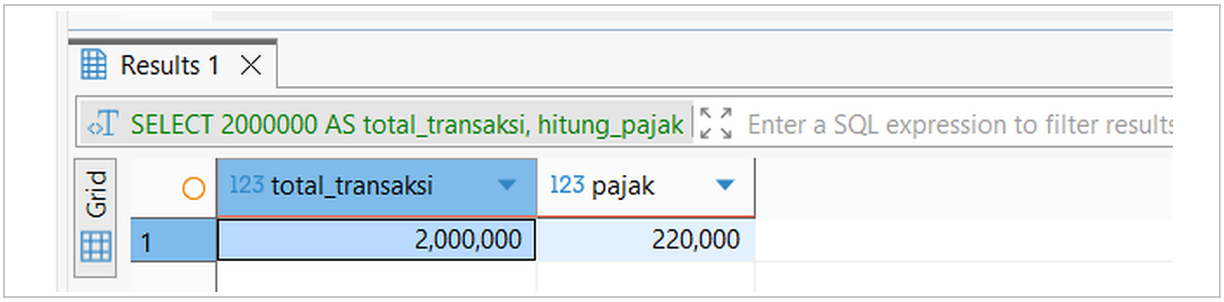
SHOW TRIGGERS FROM praktikum_function_trigger;
```

3. Bukti Eksekusi dan Hasil Pengujian

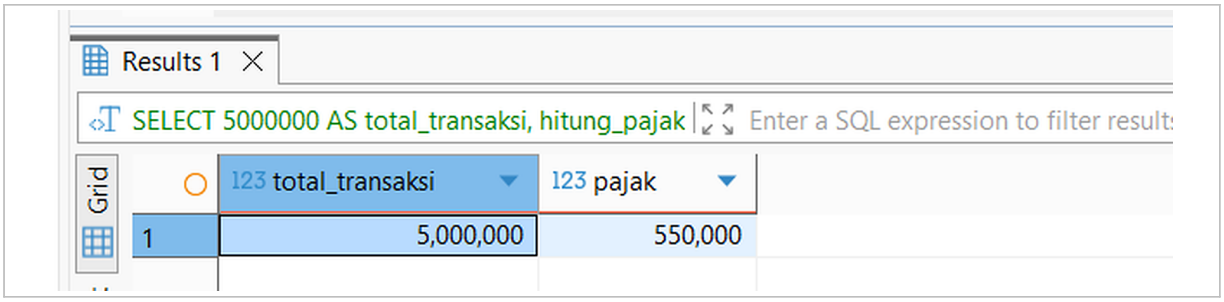
3.1 Tugas 1. Function hitung_pajak

Function diuji dengan dua total transaksi. Pajak yang dihasilkan sesuai dengan nilai yang diharapkan.

Total transaksi	Pajak yang diharapkan	Pajak hasil pengujian
Rp2.000.000	Rp220.000	Rp220.000
Rp5.000.000	Rp550.000	Rp550.000



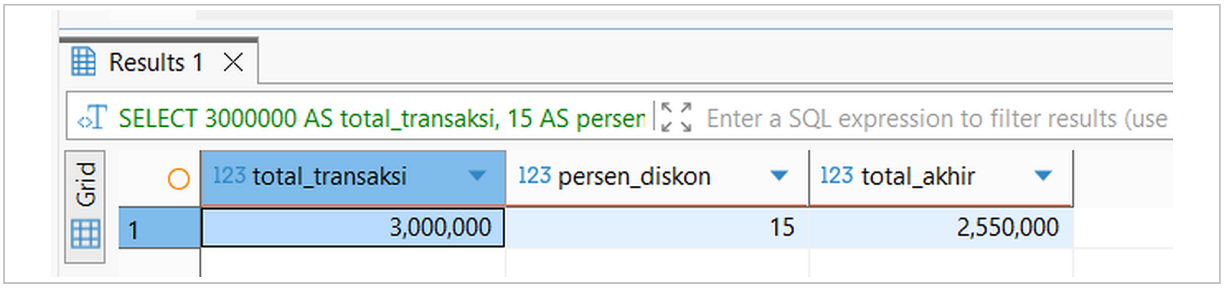
Gambar 2. Hasil pengujian function hitung_pajak: total transaksi Rp2.000.000



Gambar 3. Hasil pengujian function hitung_pajak: total transaksi Rp5.000.000

3.2 Tugas 2. Function hitung_total_akhir

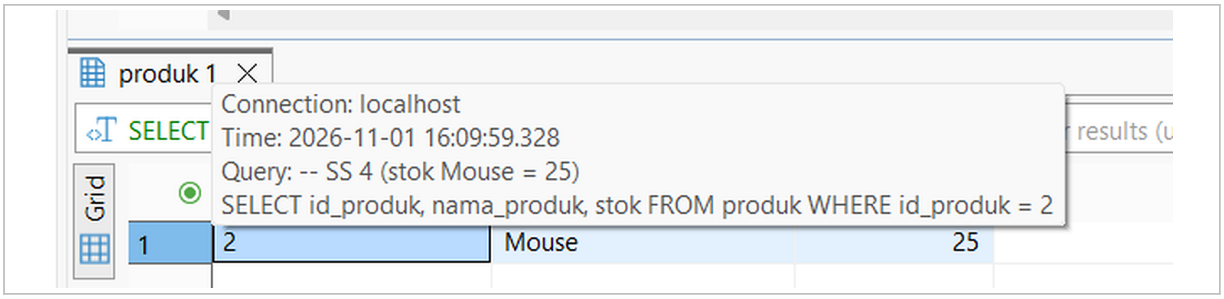
Function diuji dengan total transaksi Rp3.000.000 dan diskon 15%. Perhitungannya adalah 3.000.000 - (3.000.000 x 15 / 100) = 2.550.000, sesuai dengan target.



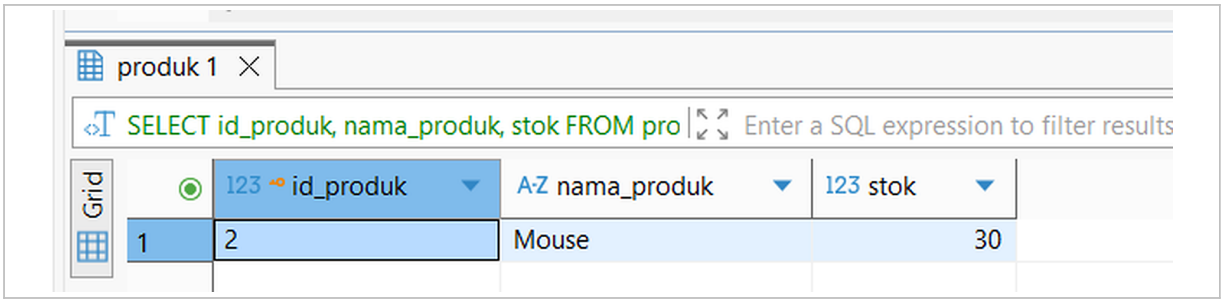
Gambar 4. Hasil pengujian function hitung_total_akhir (Rp3.000.000, diskon 15%)

3.3 Tugas 3. Trigger Pencatatan Perubahan Stok

Stok Mouse dikembalikan ke 25 unit sebelum trigger dibuat, lalu diubah menjadi 30 unit untuk menguji trigger catat_perubahan_stok.



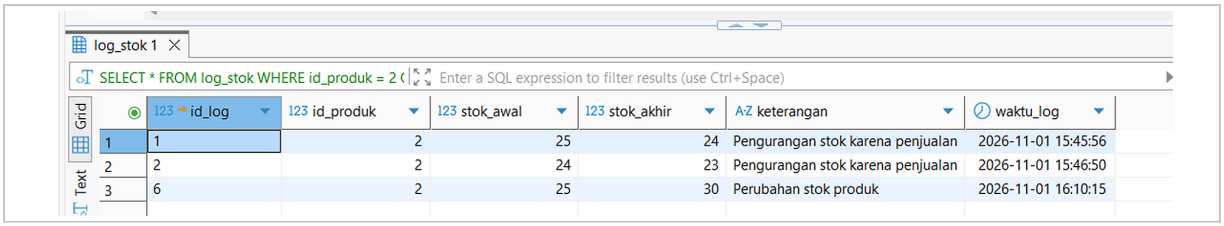
Gambar 5. Stok Mouse sebelum pengujian trigger (25 unit)



Gambar 6. Stok Mouse setelah diubah menjadi 30 unit

Setelah perubahan stok, trigger mencatat perubahan secara otomatis ke tabel log_stok dengan stok awal 25, stok akhir 30, dan keterangan "Perubahan stok produk" (id_log 6). Dua baris

sebelumnya (id_log 1 dan 2) merupakan catatan penjualan Mouse dari pengujian tugas terstruktur.


log_stok 1

SELECT * FROM log_stok WHERE id_produk = 2 (Enter a SQL expression to filter results (use Ctrl+Space)

Grid

	id_log	id_produk	stok_awal	stok_akhir	keterangan	waktu_log
1	1	2	25	24	Pengurangan stok karena penjualan	2026-11-01 15:45:56
2	2	2	24	23	Pengurangan stok karena penjualan	2026-11-01 15:46:50
3	6	2	25	30	Perubahan stok produk	2026-11-01 16:10:15

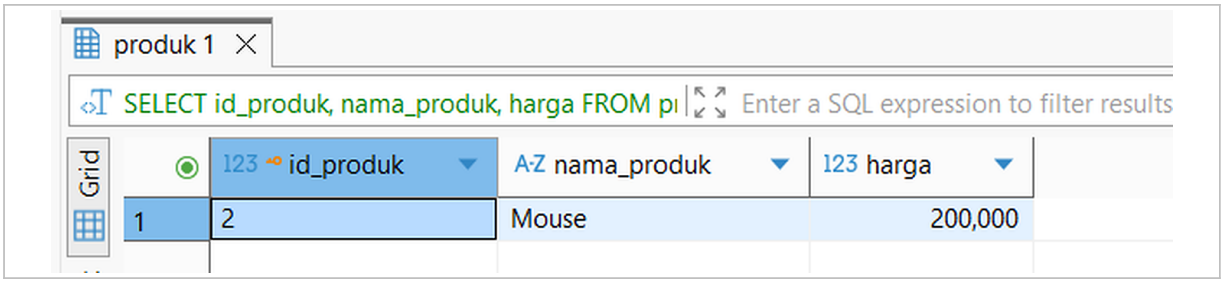
Text

Gambar 7. Isi tabel log_stok untuk produk Mouse

3.4 Tugas 4. Trigger Validasi Harga Produk

Trigger validasi_harga_produk diuji dengan dua skenario perubahan harga Mouse.

Skenario	Perubahan harga	Hasil
A (valid)	Menjadi Rp200.000	Diterima, harga Mouse menjadi Rp200.000
B (tidak valid)	Menjadi Rp0	Ditolak: "Harga produk harus lebih dari nol"

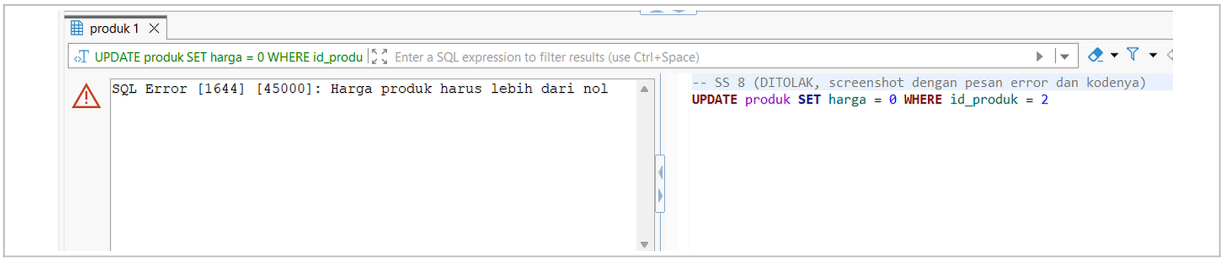

produk 1

SELECT id_produk, nama_produk, harga FROM pı Enter a SQL expression to filter results

Grid

	id_produk	nama_produk	harga
1	2	Mouse	200,000

Gambar 8. Skenario A: harga Mouse diubah menjadi Rp200.000 (diterima)


produk 1

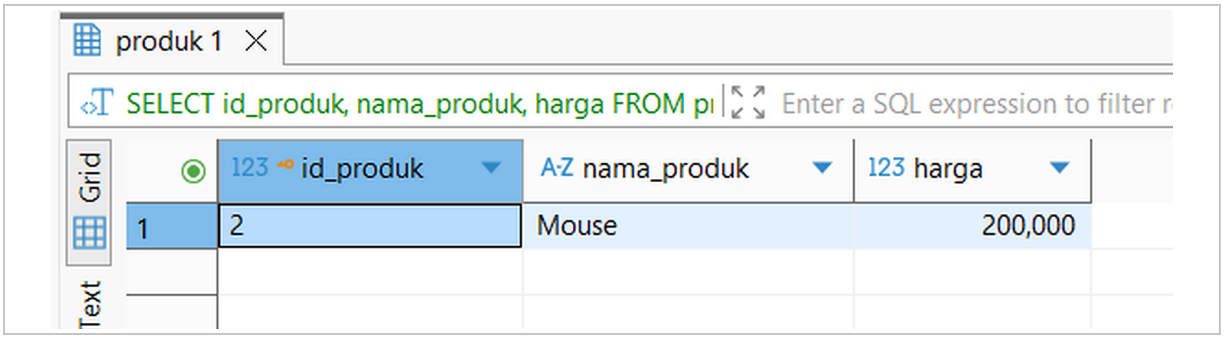
UPDATE produk SET harga = 0 WHERE id_produ Enter a SQL expression to filter results (use Ctrl+Space)

SQL Error [1644] [45000]: Harga produk harus lebih dari nol

```
-- SS 8 (DITOLAK, screenshot dengan pesan error dan kodenya)
UPDATE produk SET harga = 0 WHERE id_produk = 2
```

Gambar 9. Skenario B: perubahan harga Mouse menjadi Rp0 (ditolak)

Setelah perubahan ditolak, harga Mouse tetap Rp200.000 sehingga data tidak berubah.


produk 1

SELECT id_produk, nama_produk, harga FROM pı Enter a SQL expression to filter r

Grid

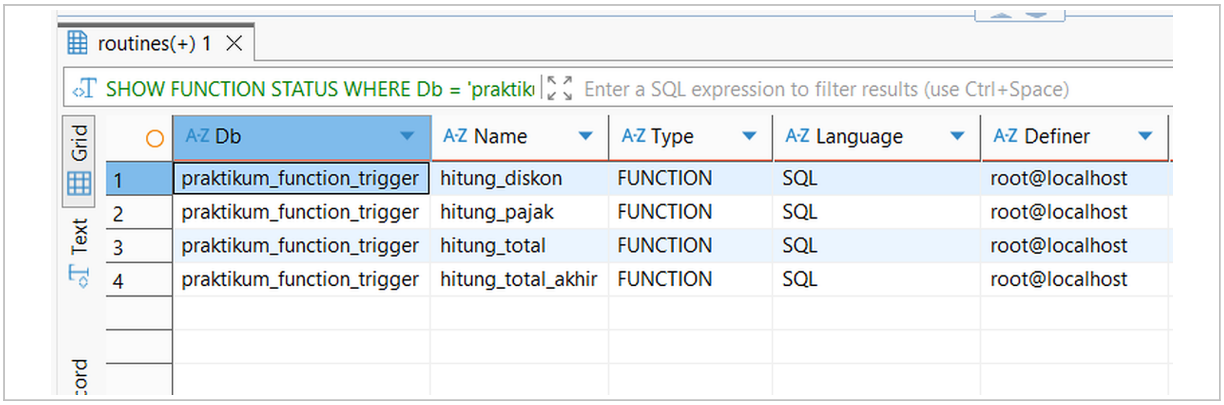
	id_produk	nama_produk	harga
1	2	Mouse	200,000

Text

Gambar 10. Harga Mouse tetap Rp200.000 setelah perubahan ditolak

3.5 Pemeriksaan Function dan Trigger

Daftar function menampilkan hitung_total, hitung_diskon, hitung_pajak, dan hitung_total_akhir. Daftar trigger menampilkan validasi_penjualan, update_stok_penjualan, validasi_harga_produk, dan catat_perubahan_stok.

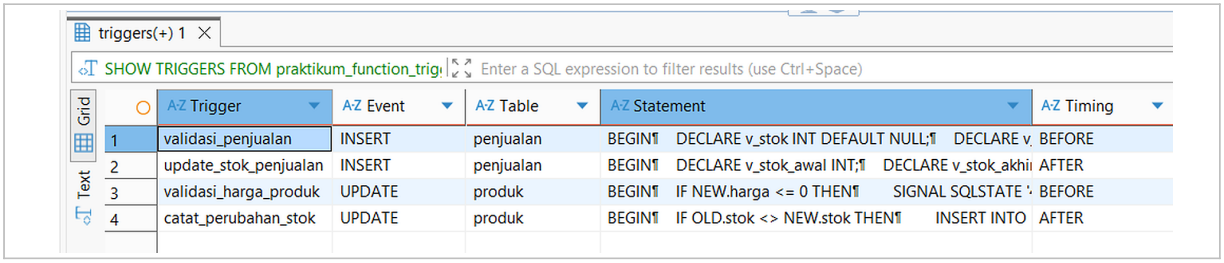

routines(+) 1

SHOW FUNCTION STATUS WHERE Db = 'praktiku | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	Db	Name	Type	Language	Definer
1	praktikum_function_trigger	hitung_diskon	FUNCTION	SQL	root@localhost
2	praktikum_function_trigger	hitung_pajak	FUNCTION	SQL	root@localhost
3	praktikum_function_trigger	hitung_total	FUNCTION	SQL	root@localhost
4	praktikum_function_trigger	hitung_total_akhir	FUNCTION	SQL	root@localhost

Gambar 11. Daftar function pada basis data praktikum_function_trigger


triggers(+) 1

SHOW TRIGGERS FROM praktikum_function_trigg | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	Trigger	Event	Table	Statement	Timing
1	validasi_penjualan	INSERT	penjualan	BEGIN¶ DECLARE v_stok INT DEFAULT NULL;¶ DECLARE v	BEFORE
2	update_stok_penjualan	INSERT	penjualan	BEGIN¶ DECLARE v_stok_awal INT;¶ DECLARE v_stok_akhi	AFTER
3	validasi_harga_produk	UPDATE	produk	BEGIN¶ IF NEW.harga <= 0 THEN¶ SIGNAL SQLSTATE '	BEFORE
4	catat_perubahan_stok	UPDATE	produk	BEGIN¶ IF OLD.stok <> NEW.stok THEN¶ INSERT INTO	AFTER

Gambar 12. Daftar trigger pada basis data praktikum_function_trigger

3.6 Ringkasan Pengujian Valid dan Tidak Valid

Pengujian	Jenis	Hasil
hitung_pajak	Valid	Rp220.000 dan Rp550.000
hitung_total_akhir	Valid	Rp2.550.000
Ubah stok Mouse 25 ke 30	Valid	Diterima dan tercatat di log_stok
Ubah harga Mouse ke Rp200.000	Valid	Diterima
Ubah harga Mouse ke Rp0	Tidak valid	Ditolak oleh trigger

4. Analisis Hasil

a. Perbedaan function hitung_total dengan trigger validasi_penjualan. Function hitung_total adalah rutin yang menerima parameter dan mengembalikan nilai, serta dipanggil secara manual di dalam query. Trigger validasi_penjualan berjalan otomatis ketika terjadi INSERT pada tabel penjualan, memvalidasi data, dan dapat menolak transaksi dengan SIGNAL. Pada praktikum ini trigger tersebut juga memanggil function hitung_total untuk mengisi kolom total.

b. Dampak jika trigger pembaruan stok tidak dibuat. Stok pada tabel produk tidak berkurang setelah penjualan sehingga data persediaan tidak sesuai kenyataan. Barang yang sebenarnya sudah habis tetap dapat terjual, dan validasi stok pada trigger validasi_penjualan menjadi tidak akurat karena membaca stok yang tidak diperbarui.

c. Alasan perubahan stok perlu dicatat dalam tabel log. Tabel log menyimpan riwayat stok awal, stok akhir, keterangan, dan waktu perubahan sehingga tersedia jejak audit. Dengan log tersebut, penyebab selisih stok dapat ditelusuri dan setiap perubahan dapat dipertanggungjawabkan, seperti terlihat pada catatan perubahan stok Mouse dari 25 menjadi 30. Perlu diperhatikan bahwa trigger update_stok_penjualan juga mengubah stok pada tabel produk, sehingga secara rancangan perubahan tersebut dapat memicu trigger catat_perubahan_stok dan menambah catatan log.

d. Peran function dan trigger dalam menjaga integritas data. Function memastikan rumus perhitungan (total, diskon, pajak) sama di setiap tempat penggunaannya. Trigger menolak data tidak valid sebelum disimpan, misalnya jumlah nol, stok tidak cukup, atau harga nol, serta menjaga stok dan log tetap sinkron secara otomatis. Dengan demikian aturan bisnis tidak bergantung pada ketelitian pengguna.

e. Hal yang perlu diperhatikan saat dua transaksi membeli produk dengan stok terbatas secara bersamaan. Kedua transaksi dapat membaca stok yang sama dan sama-sama lolos validasi (race condition) sehingga stok menjadi minus atau terjadi overselling. Pada trigger validasi_penjualan, perintah SELECT ... FOR UPDATE mengunci baris produk pada InnoDB sehingga transaksi kedua menunggu transaksi pertama selesai (COMMIT) lalu membaca stok terbaru. Oleh karena itu tabel perlu menggunakan mesin InnoDB dan transaksi dijalankan dengan benar, sedangkan CHECK (stok >= 0) menjadi pengaman terakhir.

5. Kesimpulan

- Function hitung_pajak dan hitung_total_akhir menghasilkan nilai yang sesuai target pengujian (Rp220.000, Rp550.000, dan Rp2.550.000).
- Trigger AFTER UPDATE catat_perubahan_stok berhasil mencatat perubahan stok Mouse dari 25 menjadi 30 ke tabel log_stok menggunakan OLD dan NEW.
- Trigger BEFORE UPDATE validasi_harga_produk menerima perubahan harga yang valid dan menolak perubahan harga menjadi nol.
- Kombinasi function dan trigger membantu menjaga konsistensi perhitungan, validasi data, dan integritas persediaan pada sistem penjualan.